

Treinamento de Modelos de Machine Learning para Detecção de Ransomware Baseado em Dataset Publico

Um Estudo de Caso de 4 Modelos

João Vítor Santa Brígida da Silva ¹

¹ Programa de Pós-Graduação em Computação Aplicada (PPCA)
Universidade Federal do Pará (UFPA) – Tucuruí – PA – Brasil

joao.silva@castanhal.ufpa.br

Abstract. *Ransomware detection is a critical challenge for cybersecurity, requiring solutions that identify threats in real-time based on behavior. This work presents a case study focused on the training and evaluation of four Machine Learning models: BernoulliNB, Random Forest, Logistic Regression, and KNN. Using a dynamic analysis dataset with 1,524 samples (582 ransomware and 942 goodware), the methodology employed the Variance Threshold technique to reduce high dimensionality from 30,967 to 485 essential binary attributes, such as API calls and registry operations. The models were optimized via GridSearchCV, resulting in superior performance for Logistic Regression and Random Forest, both achieving an **F1-score above 96%**. Additionally, the study explored model interpretability through native coefficients and the SHAP framework, identifying decisive behavioral patterns for threat classification.*

Resumo. *A detecção de ransomware representa um desafio crítico para a segurança cibernética, demandando soluções que identifiquem ameaças em tempo real com base em comportamento. Este trabalho apresenta um estudo de caso focado no treinamento e avaliação de quatro modelos de Machine Learning: BernoulliNB, Random Forest, Logistic Regression e KNN. Utilizando um dataset de análise dinâmica com 1.524 amostras (582 ransomware e 942 goodwares), a metodologia empregou a técnica de Variance Threshold para reduzir a alta dimensionalidade de 30.967 para 485 atributos binários essenciais, como chamadas de API e operações de registro. Os modelos foram otimizados via GridSearchCV, resultando em desempenhos superiores para a Regressão Logística e Random Forest, ambos com **F1-score acima de 96%**. Adicionalmente, o estudo explorou a interpretabilidade dos modelos através de coeficientes nativos e do framework SHAP, identificando padrões comportamentais decisivos para a classificação das ameaças.*

1. Introdução

A detecção de ameaças cibernéticas, especificamente o ransomware, representa um desafio crescente para a segurança da informação, exigindo métodos eficazes para identificar comportamentos maliciosos em tempo real. Este trabalho apresenta um estudo de caso sobre o treinamento de quatro modelos de Machine Learning voltados para a detecção de ransomware, fundamentado em um dataset público consolidado na literatura.

A abordagem adotada prioriza a análise dinâmica, que consiste na execução de amostras em ambientes controlados, como o [Cuckoo Foundation 2014]. Diferente da análise estática, a análise dinâmica permite capturar o comportamento real do código em execução, mitigando técnicas de evasão comuns, como a ofuscação e a criptografia de binários. O dataset utilizado, disponibilizado por [Sgandurra et al. 2016], compreende um total de 1.524 amostras, sendo 582 (38,19%) de ransomware e 942 (61,81%) de *goodwares* (aplicações legítimas).

Um aspecto central deste estudo é a engenharia de atributos, fundamentada em 30.967 características binárias que representam a presença ou ausência de ações específicas durante a execução. Esses atributos estão organizados em sete categorias comportamentais críticas:

- **API:** Chamadas de sistema;
- **REG:** Operações em chaves de registro do Windows;
- **FILES e FILES EXT:** Operações e extensões de arquivos;
- **DROP, DIR e STR:** Criação de arquivos, operações em diretórios e strings incorporadas.

O objetivo principal deste artigo é avaliar o desempenho dos algoritmos **BernoulliNB**, **Random Forest**, **Logistic Regression** e **KNN** na classificação dessas ameaças. Além da busca pela alta acurácia e otimização de hiperparâmetros via **GridSearchCV**, o trabalho dedica-se à interpretabilidade e explicabilidade dos modelos. Através de coeficientes nativos e do framework SHAP, busca-se compreender quais chamadas de API e operações de sistema são determinantes para a identificação de um ransomware.

2. Materiais e Métodos

Nesta seção, detalham-se os procedimentos e ferramentas utilizados para a obtenção e análise inicial dos dados que fundamentam este estudo.

2.1. Descrição do Dataset e Coleta de Dados

A base de dados selecionada para este trabalho consiste em eventos de execução em sandbox (análise dinâmica), especificamente o dataset público disponibilizado por [Sgandurra et al. 2016]. Esta escolha justifica-se pelo fato de o dataset ser bem referenciado em estudos de segurança e inteligência artificial, servindo como base consolidada para a análise de comportamentos maliciosos [Moreira et al. 2022].

2.1.1. A. Natureza e Origem dos Dados

Os dados foram gerados por meio da execução de amostras em um ambiente controlado utilizando a ferramenta *Cuckoo Sandbox* [Cuckoo Foundation 2014]. Ao contrário da análise estática, a análise dinâmica permite capturar o comportamento real do código em execução, mitigando técnicas de evasão como ofuscação e criptografia de binários [Sgandurra et al. 2016]. O dataset é composto por um total de 1.524 amostras, divididas em:

- **582 malwares (Ransomware):** Representando 38,19% do total.
- **942 goodwares (Aplicações legítimas):** Representando 61,81% do total.

As amostras de ransomware estão subdivididas em 11 famílias distintas, permitindo uma análise granular dos diferentes padrões de ataque [Sgandurra et al. 2016]. Entre as famílias presentes, destacam-se *CryptLocker* (107 amostras), *Locker* (97) e *Reveton* (90), como é listado mais detalhadamente na tabela 1.

2.1.2. B. Variáveis e Engenharia de Atributos (Features)

O dataset original contém 30.967 atributos binários, que representam a presença (1) ou ausência (0) de uma determinada ação ou evento durante a execução[Sgandurra et al. 2016]. Para este estudo foi inicialmente desenvolvido um script em python (Figura 1) para o levantamento estatístico inicial do dataset (mais detalhado na Tabela 1). Estes atributos estão organizados em sete categorias de comportamento:

- **API:** Invocações de chamadas de sistema (*API calls*).
- **DROP:** Extensões de arquivos criados (*dropped*) durante a execução.
- **REG:** Operações em chaves de registro do Windows.
- **FILES:** Operações gerais de sistema de arquivos (leitura, escrita, deleção).
- **FILES_EXT (EXT):** Extensões dos arquivos envolvidos nas operações de arquivo.
- **DIR:** Operações em diretórios (criação, enumeração).
- **STR:** Strings incorporadas no binário original.

2.2. Estatísticas do Dataset

A Tabela 1 apresenta a distribuição detalhada das amostras de ransomware por família, conforme extraído do processamento dos dados brutos.

3. Pre-Processamento

Este capítulo detalha as etapas de preparação do conjunto de dados, abordando a limpeza de dados, a seleção de atributos e a fundamentação teórica que sustenta as escolhas técnicas realizadas para otimizar o treinamento dos modelos de detecção de ransomware.

3.1. Limpeza de Dados e Seleção de Atributos via Variance Threshold

O conjunto de dados original apresentava uma alta dimensionalidade, com um total de 30.967 atributos comportamentais. No entanto, observou-se que a vasta maioria desses atributos (98,46%) possuía uma baixíssima densidade de informação, apresentando entre 0 e 50 eventos em um universo de 1.524 amostras. Para reduzir este problema de esparsidade e minimizar o custo computacional, aplicou-se a técnica de seleção de atributos *Variance Threshold* (VT).

O VT é um método do tipo filtro não supervisionado que remove recursos cuja variância não atinge um limite especificado [Kuhn and Johnson 2013]. Dado que o *dataset* é composto por variáveis binárias (booleanas), a variância é calculada com base no ensaio de Bernoulli, conforme a Equação 1:

$$Var[X] = p(1 - p) \quad (1)$$

Equação para o cálculo da variância.

```

1 import pandas as pd
2
3 # Carrega os nomes de famílias
4 # 'Family Names ID.txt' tem formato de tabela textual, é tratado os espaços
5 family_mapping = {}
6 with open('Family Names ID.txt', 'r', encoding='utf-8') as f:
7     for line in f:
8         line = line.strip()
9         # Ignora cabeçalhos ou linhas vazias
10        if not line or 'FAMILY NAME' in line or '---' in line:
11            continue
12
13        # Divide a linha baseado no ID numérico que fica no final
14        parts = line.rsplit(maxsplit=1)
15        if len(parts) == 2:
16            name, idx = parts[0].strip().replace("'", ""), int(parts[1])
17            family_mapping[idx] = name
18
19 # Carrega dataset com os headers configurados
20 df = pd.read_csv('RansomwareData_Com_Headers.csv', low_memory=False)
21
22 # Identifica nomes exatos das colunas com base no VariableNames.txt
23 col_label = 'Label (1 Ransomware / 0 Goodware)'
24 col_family = 'Ransomware Family'
25
26 # Remove as colunas de ID, Label e Family
27 num_features = df.shape[1] - 3
28
29 # --- Estatísticas Automáticas ---
30 total = len(df)
31 malware = df[df[col_label] == 1].shape[0]
32 goodware = df[df[col_label] == 0].shape[0]
33 familias_unicas = df[col_family].nunique()
34
35 # --- Abre um .txt para salvar os resultados ---
36 with open('resultado_estatisticas.txt', 'w', encoding='utf-8') as arquivo_saida:
37
38     # Adiciona argumento file=arquivo_saida em cada print
39     print(f"Total de Amostras: {total}", file=arquivo_saida)
40     print(f"Ransomwares: {malware} ({(malware/total)*100:.2f}%)", file=arquivo_saida)
41     print(f"Goodwares: {goodware} ({(goodware/total)*100:.2f}%)", file=arquivo_saida)
42     print(f"Famílias Únicas Detectadas: {familias_unicas}", file=arquivo_saida)
43     print(f"Total de Atributos Computados (Features): {num_features}", file=arquivo_saida)
44
45     # --- Detalhamento por Família ---
46     print("\n--- Amostras por Família de Ransomware ---", file=arquivo_saida)
47     family_counts = df[col_family].value_counts()
48
49     for idx, count in family_counts.items():
50         nome_familia = family_mapping.get(idx, f"ID Desconhecido ({idx})")
51         print(f"- {nome_familia:<20}: {count} amostras", file=arquivo_saida)
52
53 print("O resultado foi salvo com sucesso em 'resultado_estatisticas.txt'!")
54

```

Figura 1. Script Python para levantamento estatístico do dataset.

Tabela 1. Visão Geral e Distribuição de Amostras do Dataset Sgandurra (2016).

Métrica	Valor / Quantidade
Total de Amostras	1.524
Ransomwares	582 (38.19%)
Goodwares	942 (61.81%)
Famílias Únicas Detectadas	11
Total de Atributos Computados (Features)	30.967
Amostras por Família de Ransomware	
Goodware	942
CryptLocker	107
Locker	97
Reveton	90
Kovter	64
MATSNU	59
Critroni	50
CryptoWall	46
Trojan-Ransom	34
KOLLAH	25
TeslaCrypt	6
PGPCODER	4

Onde p representa a probabilidade de sucesso (parâmetro de probabilidade de Bernoulli), que define o percentual do limite de corte [Pedregosa et al. 2011]. Neste trabalho, utilizou-se um limite de variância de 97% ($p = 0.97$). Na prática, isso resultou na exclusão de todas as colunas onde o mesmo valor (0 ou 1) se repetia em mais de 97% das amostras, ou seja, atributos com 45 ou menos ocorrências de um dos valores foram descartados por não serem considerados cruciais. Esta etapa reduziu drasticamente o número de atributos de 30.923 para 485, permitindo uma análise mais focada em características realmente discriminantes.

3.2. Normalização e Mapeamento de Atributos

Quanto à normalização, o conjunto de dados já se encontrava modelado em formato binário, onde o valor '1' indica a presença de uma operação ou evento e '0' indica sua ausência. Devido a essa natureza booleana intrínseca, não foi necessária a aplicação de técnicas de reescalonamento numérico adicionais.

Uma tarefa essencial de pré-processamento foi a atribuição de nomes às *features*. Como o *dataset* público original não possuía cabeçalhos nas colunas, foi realizado o mapeamento técnico de cada atributo utilizando um arquivo de metadados separado. Esta ação foi fundamental para garantir a interpretabilidade dos resultados, permitindo identificar quais chamadas de API ou operações de registro são mais frequentes em amostras de ransomware em comparação a *goodwares*.

3.3. Fundamentação Teórica

A escolha pelo *Variance Threshold* justifica-se por ser uma abordagem não supervisionada de filtro. Diferente de métodos supervisionados, o VT pode ser aplicado ao conjunto

de dados global antes da estratégia de validação cruzada (*Cross-validation*), garantindo que os mesmos atributos sejam mantidos em todas as iterações de treino e teste sem o risco de *overfitting* [James et al. 2013]. Além disso, como um método de filtragem, o VT preserva o significado original das variáveis, o que é vital para a análise comportamental de *malwares* [Pedregosa et al. 2011].

A literatura de aprendizado de máquina sustenta que o uso de distribuições de Bernoulli para o cálculo de variância em dados binários é a abordagem estatística correta para identificar recursos com baixa variabilidade [Hastie et al. 2009]. Estudos anteriores utilizando este mesmo *dataset* demonstraram que a redução de dimensionalidade para cerca de 400-500 atributos mantém ou até melhora a acurácia dos classificadores, validando a eficácia do limite de 97% adotado [Sgandurra et al. 2016, Moreira et al. 2022].

4. Treinamento dos Modelos de Machine Learning

Este capítulo descreve o processo de treinamento e otimização dos modelos de *Machine Learning* utilizados para a detecção de *ransomware*. O objetivo principal é classificar o *dataset* entre as categorias *goodware* e *malware*, empregando técnicas de aprendizado supervisionado utilizando Python, com o auxílio da biblioteca *Scikit-learn*.

4.1. Seleção de Modelos e Hiperparâmetros

Para este estudo, optou-se pela abordagem de **aprendizado supervisionado**, sendo selecionados quatro algoritmos classificadores distintos para avaliação:

- **BernoulliNB**: Um classificador Naive Bayes adequado para dados discretos. Os hiperparâmetros pesquisados incluíram *alpha*, *binarize* e *fit_prior*.
- **Random Forest**: Um método de aprendizado de conjunto (*ensemble learning*). Foram analisados os parâmetros *class_weight*, *criterion*, *max_depth* e *n_estimators*.
- **Logistic Regression**: Utilizado para classificação linear. A pesquisa de hiperparâmetros focou em *C*, *max_iter*, *penalty* e *solver*.
- **KNeighborsClassifier (KNN)**: Baseado na proximidade de vizinhos. Os parâmetros ajustados foram *metric*, *n_neighbors* e *weights*.

4.2. Otimização com GridSearchCV

Para maximizar o desempenho dos modelos, especificamente a métrica **F1-score**, foi realizado o ajuste de hiperparâmetros utilizando a técnica de *GridSearchCV* com validação cruzada, utilizando **Folds = 5** e **Repetições = 10**. Este processo permitiu identificar a melhor configuração para cada modelo, conforme detalhado na Tabela 2.

4.3. Resultados e Discussão

Após a identificação dos melhores hiperparâmetros, os modelos foram reavaliados. A Tabela 3 apresenta os resultados comparativos de desempenho, considerando as métricas de Acurácia, Precisão, *Recall* e F1-score, acompanhadas de seus respectivos intervalos de confiança de 95%.

Os resultados demonstram que os modelos de **Logistic Regression** e **Random Forest** apresentaram os melhores desempenhos gerais, ambos superando a marca de 96% em F1-score após a otimização. O modelo KNN também apresentou um desempenho sólido (92,59% F1-score), enquanto o BernoulliNB, embora eficaz, obteve métricas inferiores aos demais classificadores testados.

Tabela 2. Melhores Hiperparâmetros Identificados via GridSearchCV

Modelo	Melhores Parâmetros Configurativos
BernoulliNB	{'alpha': 0.001, 'binarize': None, 'fit_prior': False}
KNN	{'metric': 'manhattan', 'n_neighbors': 3, 'weights': 'distance'}
Logistic Regression	{'C': 1.0, 'max_iter': 1000, 'penalty': 'l1', 'solver': 'saga'}
Random Forest	{'class_weight': 'None', 'criterion': 'entropy', 'max_depth': 20, 'n_estimators': 100}

Tabela 3. Comparativo de Desempenho dos Modelos Otimizados

Modelo	Acurácia (%)	Precisão (%)	Recall (%)	F1-score (%)
BernoulliNB	82,09 ± 0,57	71,77 ± 0,78	87,80 ± 0,79	78,94 ± 0,62
KNN	95,58 ± 0,23	93,99 ± 0,62	94,57 ± 0,54	94,24 ± 0,29
Logistic Regression	97,69 ± 0,23	97,35 ± 0,37	96,60 ± 0,43	96,96 ± 0,30
Random Forest	97,82 ± 0,20	97,58 ± 0,36	96,70 ± 0,44	97,12 ± 0,26

5. Interpretando o Modelo e Análise de Influência das Features

Este capítulo descreve o processo de interpretação dos resultados obtidos pelo modelo selecionado para a detecção de malwares. A interpretabilidade é um componente crítico em segurança cibernética, pois permite não apenas a classificação, mas a compreensão dos comportamentos que levam o modelo a identificar uma ameaça.

5.1. Seleção e Treinamento do Modelo

Com base em experimentos prévios, o modelo de **Regressão Logística** foi o escolhido para a análise final. Esta decisão fundamentou-se no fato de o modelo apresentar resultados de acurácia próximos aos do Random Forest, porém com um desempenho de processamento e eficiência computacional consideravelmente superiores. O treinamento foi realizado utilizando os seguintes hiperparâmetros otimizados:

- Penalidade: *L1* Lasso (Least Absolute Shrinkage and Selection Operator);
- Parâmetro de Regularização (*C*): 1.0;
- Solucionador: *saga*;
- Máximo de iterações: 1000.

5.2. Cálculo Nativo de Importância (Coeficientes)

A Regressão Logística, por ser um modelo linear, permite uma interpretação direta através de seus coeficientes [Hosmer et al. 2013]. Cada variável de entrada (feature) recebe um peso que representa sua contribuição para o log-odds da classe alvo.

A Figura 2 apresenta o *Top 30 APIs por Importância Nativa*. No gráfico:

- **Coeficientes Positivos (Vermelho):** Indicam que a presença ou alta frequência da API aumenta a probabilidade de a amostra ser classificada como *Ransomware*. Destacam-se APIs relacionadas a manipulação de strings e acesso a registros.
- **Coeficientes Negativos (Azul):** Indicam uma associação com *Goodware*. A API `STR:5346` e `API:NtReadVirtualMemory` apresentam os maiores pesos negativos, sugerindo que são características predominantes em softwares legítimos no dataset analisado.

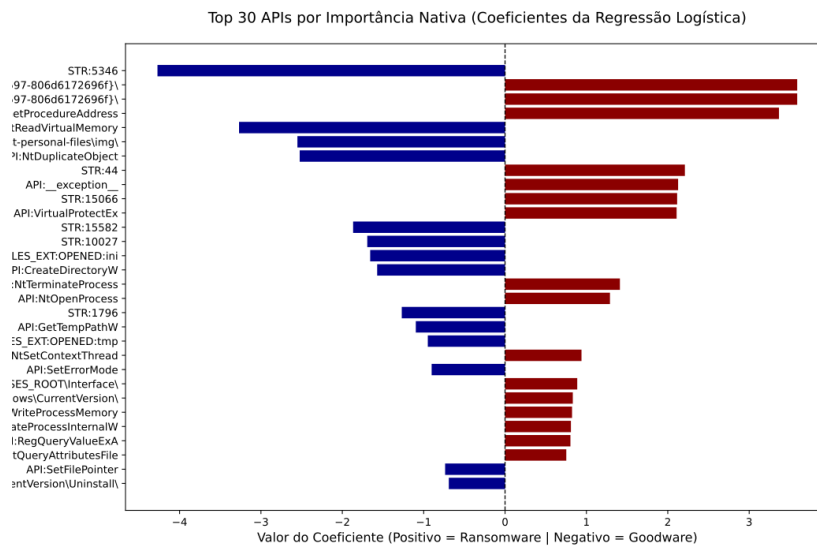


Figura 2. Gráfico do Cálculo Nativo do modelo Logistic Regression em seus melhores parâmetros.

5.3. Análise de Explicabilidade com SHAP

Para aprofundar a compreensão sobre a influência das features, utilizamos o framework SHAP (*SHapley Additive exPlanations*), fundamentado na teoria dos jogos cooperativos [Lundberg and Lee 2017]. O SHAP oferece uma visão mais granular do que os coeficientes globais, permitindo observar o impacto de cada feature em cada predição individual.

5.3.1. Beeswarm Plot

O gráfico *Beeswarm* (Figura 3) fornece uma visão geral do impacto das top 30 APIs no modelo.

- O eixo horizontal representa o **Impacto SHAP**: valores à direita aumentam a probabilidade de *Ransomware*, enquanto à esquerda diminuem.
- A cor indica o valor da feature (Vermelho para alto, Azul para baixo).
- Observa-se que para a API `NtReadVirtualMemory`, valores baixos (azul) têm um impacto negativo forte, corroborando o gráfico de coeficientes nativos, enquanto para outras APIs, a dispersão dos pontos revela comportamentos não lineares capturados pelo modelo.

5.3.2. Decision Plot

O gráfico *Decision Plot* (Figura 4) ilustra o caminho de decisão para 50 amostras específicas.

- O gráfico mostra como as predições individuais evoluem a partir de um valor base (valor médio esperado) conforme cada API é considerada.

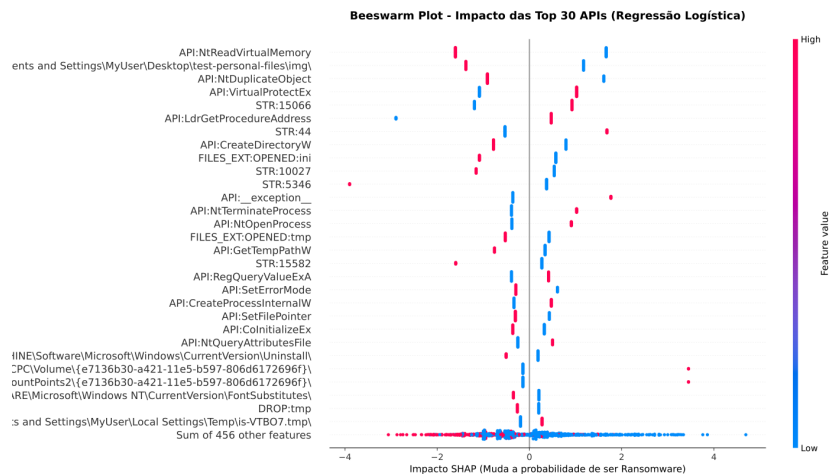


Figura 3. Grafico SHAP Beeswarm com o modelo Logistic Regression em seus melhores parametros.

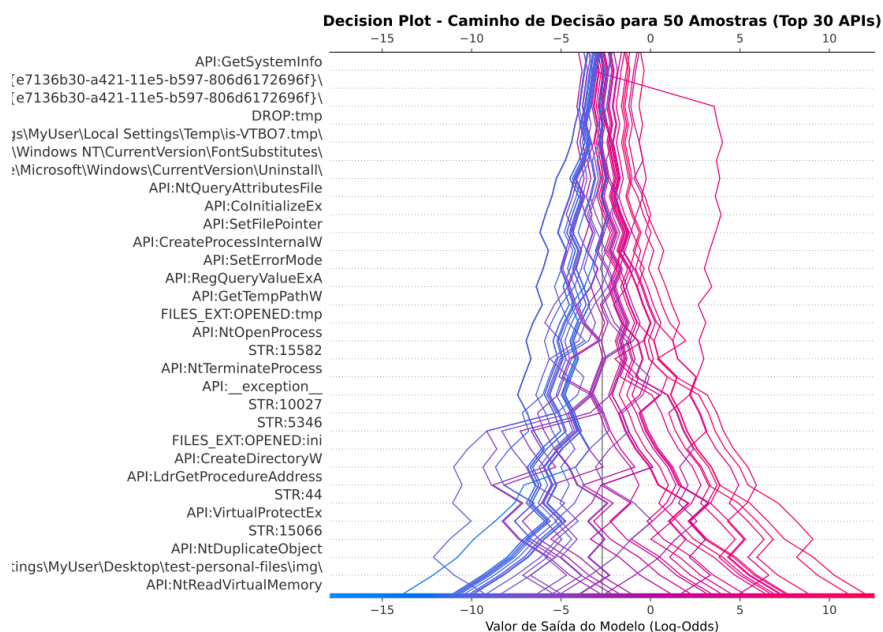


Figura 4. Grafico SHAP Decision Plot com o modelo Logistic Regression em seus melhores parametros.

- As linhas que convergem para a direita (em rosa) representam amostras com alta probabilidade de serem maliciosas, enquanto as que tendem para a esquerda (em azul) são classificadas como benignas.
- Este gráfico é essencial para identificar quais combinações de APIs são determinantes para a classificação final do modelo em casos específicos de *log-odds*.

6. Conclusão

Este estudo realizou uma análise comparativa de quatro modelos de *Machine Learning* para a detecção de ransomware, utilizando um dataset baseado em análise dinâmica com 1.524 amostras. Os resultados demonstram a viabilidade e a eficácia de utilizar o comportamento em execução para diferenciar códigos maliciosos de aplicações legítimas (*goodwares*).

A etapa de pré-processamento, especificamente a aplicação da técnica de *Variance Threshold*, mostrou-se fundamental. A redução drástica da dimensionalidade de 30.967 para 485 atributos não apenas diminuiu o custo computacional, mas manteve a capacidade discriminante necessária para o treinamento dos modelos. Entre os algoritmos avaliados, a Regressão Logística e o Random Forest destacaram-se com desempenhos superiores, ambos alcançando um F1-score acima de 96%.

A escolha da Regressão Logística para a análise final foi justificada por seu equilíbrio entre alta acurácia e eficiência computacional. Além da detecção, este trabalho contribuiu para a explicabilidade do modelo ao utilizar coeficientes nativos e o framework SHAP. Essa análise permitiu identificar que chamadas de API específicas e operações de sistema, como as relacionadas à manipulação de strings e acesso a registros, são indicadores críticos de comportamento de ransomware no dataset estudado.

Em suma, a metodologia adotada prova que a combinação de engenharia de atributos focada em comportamento e modelos de aprendizado supervisionado é uma abordagem robusta para enfrentar as ameaças cibernéticas modernas. Como trabalhos futuros, sugere-se a expansão do dataset para incluir famílias de ransomware mais recentes e a exploração de métodos de aprendizado profundo (*Deep Learning*) para capturar sequências temporais de chamadas de sistema.

Referências

- Cuckoo Foundation (2014). Automated malware analysis - cuckoo sandbox. Acessado em: 31 de Maio de 2026.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer New York, 2nd edition.
- Hosmer, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied Logistic Regression*. John Wiley & Sons.
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Springer Science & Business Media.
- Kuhn, M. and Johnson, K. (2013). *Applied Predictive Modeling*. Springer, New York.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

- Moreira, C. C., Sales, Jr., C. d. S. d., and Moreira, D. C. (2022). Understanding ransomware actions through behavioral feature analysis. *Journal of Communication and Information Systems*, 37(1):61–76.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). *Scikit-learn: Machine Learning in Python*.
- Sgandurra, D., Muñoz-González, L., Mohsen, R., and Lupu, E. C. (2016). Automated Dynamic Analysis of Ransomware: Benefits, Limitations and use for Detection. *arXiv preprint arXiv:1609.03020*.